Machine-Level Programming I: Lab Session

CSE 238/2038/2138: Systems Programming

Instructors:
Lokman ALTIN & Zuhal ALTUNTAŞ

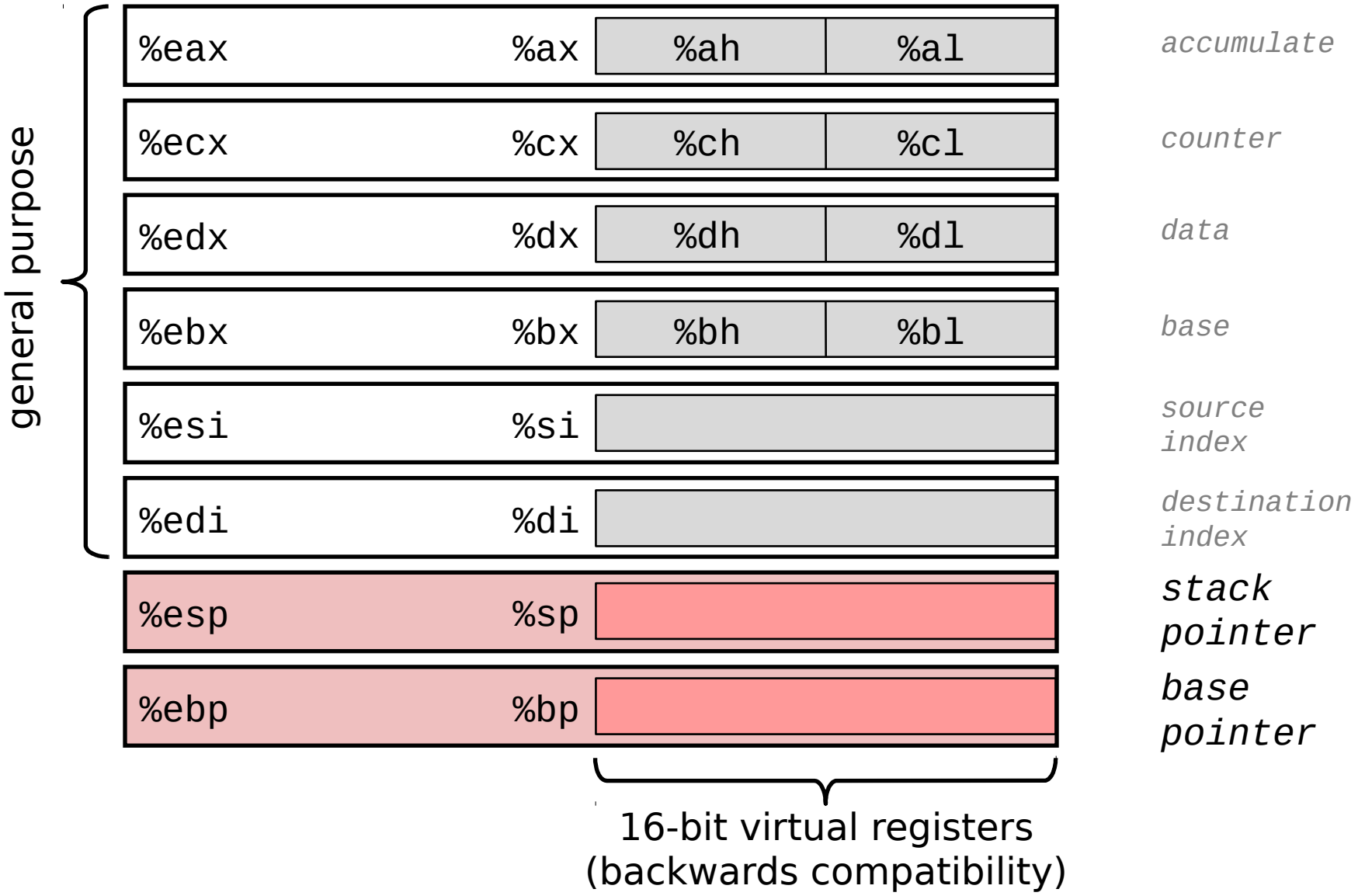
Slides adapted from Bryant & O'Hallaron's slides

x86-64 Integer Registers

%rax	%eax	%r8	%r8d
%rbx	%ebx	%r9	%r9d
%rcx	%ecx	%r10	%r10d
%rdx	%edx	%r11	%r11d
%rsi	%esi	%r12	%r12d
%rdi	%edi	%r13	%r13d
%rsp	%esp	%r14	%r14d
%rbp	%ebp	%r15	%r15d

- Can reference low-order 4 bytes (also low-order 1 & 2 bytes)
- Not part of memory (or cache)

Some History: IA32 Registers



Assembly Characteristics: Operations

- Transfer data between memory and register
  - Load data from memory into register
  - Store data from register into memory
- Perform arithmetic function on register or memory data
- Transfer control
  - Unconditional jumps to/from procedures
  - Conditional branches
  - Indirect branches

Sizes of Data Types and Code Suffix

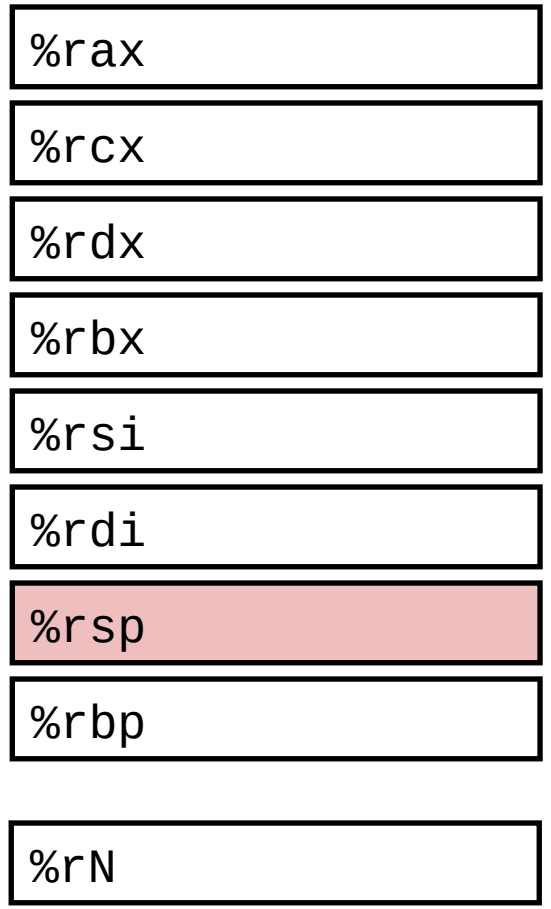
C Decleration	Intel Data Type	Assembly Code Suffix	Size (Bytes)
char	Byte	b	1
short	Word	w	2
int	Double word	l	4
long	Quad Word	q	8
char *	Quad Word	q	8
float	Single Precision	s	4
double	Double Precision	l	8

Moving Data

- Moving Data

  `movq` *Source*, *Dest*:

- Operand Types
  - ***Immediate:*** Constant integer data
    - Example: **`$0x400, $-533`**
    - Like C constant, but prefixed with **`'$'`**
    - Encoded with 1, 2, or 4 bytes
  - ***Register:*** One of 16 integer registers
    - Example: **`%rax, %r13`**
    - But **`%rsp`** reserved for special use
    - Others have special uses for particular instructions
  - ***Memory:*** 8 consecutive bytes of memory at address given by register
    - Simplest example: **`(%rax)`**
    - Various other "address modes"



Warning: Intel docs use mov *Dest, Source*

movq Operand Combinations



Cannot do memory-memory transfer with a single instruction

Simple Memory Addressing Modes

- Normal (R) Mem[Reg[R]]
  - Register R specifies memory address
  - Pointer dereferencing in C

```
movq (%rcx),%rax
```

- Displacement D(R) Mem[Reg[R]+D]
  - Register R specifies start of memory region
  - Constant displacement D specifies offset

```
movq 8(%rbp),%rdx
```

Complete Memory Addressing Modes

- Most General Form

  D(Rb,Ri,S) Mem[Reg[Rb]+S*Reg[Ri]+ D]

  - D: Constant “displacement” 1, 2, or 4 bytes
  - Rb: Base register: Any of 16 integer registers
  - Ri: Index register: Any, except for **`%rsp`**
  - S: Scale: 1, 2, 4, or 8 (*why these numbers?*)

- Special Cases

  (Rb,Ri) Mem[Reg[Rb]+Reg[Ri]]

  D(Rb,Ri) Mem[Reg[Rb]+Reg[Ri]+D]

  (Rb,Ri,S) Mem[Reg[Rb]+S*Reg[Ri]]

Address Computation Examples

%rdx	0xf000
%rcx	0x0100

D(Rb,Ri,S) **Mem[Reg[Rb]+S*Reg[Ri]+ D]**

- D: Constant "displacement" 1, 2, or 4 bytes
- Rb: Base register: Any of 16 integer registers
- Ri: Index register: Any, except for `%rsp`
- S: Scale: 1, 2, 4, or 8 (*why these numbers?*)

Expression	Address Computation	Address
0x8(%rdx)	0xf000 + 0x8	0xf008
(%rdx,%rcx)	0xf000 + 0x100	0xf100
(%rdx,%rcx,4)	0xf000 + 4*0x100	0xf400
0x80(,%rdx,2)	2*0xf000 + 0x80	0x1e080

Address Computation Instruction

- leaq Src, Dst
  - Src is address mode expression
  - Set Dst to address denoted by expression

- Uses
  - Computing addresses without a memory reference
    - E.g., translation of `p = &x[i];`
  - Computing arithmetic expressions of the form **x + k*y**
    - k = 1, 2, 4, or 8

Example

```
long m12(long x){
  return x*12;
}
```

Converted to ASM by compiler:

```
leaq (%rdi,%rdi,2), %rax # t <- x+x*2
salq $2, %rax            # return t<<2
```

Some Arithmetic Operations

- Two Operand Instructions:

Format		Computation	
`addq`	`Src,Dest`	Dest = Dest + Src	
`subq`	`Src,Dest`	Dest = Dest - Src	
`imulq`	`Src,Dest`	Dest = Dest * Src	
`salq`	`Src,Dest`	Dest = Dest << Src	-> Also called shlq
`sarq`	`Src,Dest`	Dest = Dest >> Src	-> Arithmetic right shift
`shrq`	`Src,Dest`	Dest = Dest >> Src	-> Logical right shift
`xorq`	`Src,Dest`	Dest = Dest ^ Src	
`andq`	`Src,Dest`	Dest = Dest & Src	
`orq`	`Src,Dest`	Dest = Dest | Src	

- Watch out for argument order!

  (Warning: Intel docs use “op Dest,Src”)

- No distinction between signed and unsigned int (why?)

Some Arithmetic Operations

- One Operand Instructions

```
incq   Dest    Dest = Dest + 1
decq   Dest    Dest = Dest - 1
negq   Dest    Dest = - Dest
notq   Dest    Dest = ~Dest
```

- See book for more instructions

Arithmetic Expression Example

```
long arith
(long x, long y, long z){
  long t1 = x+y;
  long t2 = z+t1;
  long t3 = x+4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```

```
arith:
  leaq    (%rdi,%rsi), %rax
  addq    %rdx, %rax
  leaq    (%rsi,%rsi,2), %rdx
  salq    $4, %rdx
  leaq    4(%rdi,%rdx), %rcx
  imulq   %rcx, %rax
  ret
```

Interesting Instructions

- **`leaq`**: address computation
- **`salq`**: shift
- **`imulq`**: multiplication
  - But, only used once

Understanding Arithmetic Expression Example

```
long arith
(long x, long y, long z){
  long t1 = x+y;
  long t2 = z+t1;
  long t3 = x+4;
  long t4 = y * 48;
  long t5 = t3 + t4;
  long rval = t2 * t5;
  return rval;
}
```

```
arith:
  leaq    (%rdi,%rsi), %rax    # t1
  addq    %rdx, %rax           # t2
  leaq    (%rsi,%rsi,2), %rdx
  salq    $4, %rdx             # t4
  leaq    4(%rdi,%rdx), %rcx   # t5
  imulq   %rcx, %rax           # rval
  ret
```

Register	Use(s)
%rdi	Argument **x**
%rsi	Argument **y**
%rdx	Argument **z**, **t4**
%rax	**t1**, **t2**, **rval**
%rcx	**t5**

Compiler optimization:

- Reuse of registers
- Substitution (copy propagation)
- Strength reduction

Compiling Into Assembly

C Code (sum.c)

```
int plus(int x, int y){
    return x+y;
}
void sumstore(int x, int y, int *dest){
    int t = plus(x, y);
    *dest = t;
}
```

Generated x86-64

```
sumstore:
    pushq   %rbp
    movq    %rsp, %rbp
    addl    %esi, %edi
    movl    %edi, (%rdx)
    popq    %rbp
    ret
```

Obtain (on Mac OS) with command

```
gcc –O –S sum.c
```

Produces file `sum.s`

Warning: Will get very different results on different machines (Linux, Mac OS-X, …) due to different versions of gcc and different compiler settings.

What it Really Looks Like

```
    .globl  _sumstore
    .align  4, 0x90
_sumstore:
    .cfi_startproc
## BB#0:
    pushq   %rbp
Ltmp3:
    .cfi_def_cfa_offset 16
Ltmp4:
    .cfi_offset %rbp, -16
    movq    %rsp, %rbp
Ltmp5:
    .cfi_def_cfa_register %rbp
    addl    %esi, %edi
    movl    %edi, (%rdx)
    popq    %rbp
    retq
    .cfi_endproc
```

Things preceding with a "." are generally directives

```
sumstore:
   pushq   %rbp
   movq    %rsp, %rbp
   addl    %esi, %edi
   movl    %edi, (%rdx)
   popq    %rbp
   ret
```

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers as in table below.
- According to given information, fill the table on the right side

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax		
0x104		
$0x108		
(%rax)		
4(%rax)		
9(%rax, %rdx)		
260(%rcx,%rdx)		
0xFC(,%rcx,4)		
(%rax, %rdx,4)		

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax	0x100	Register
0x104		
$0x108		
(%rax)		
4(%rax)		
9(%rax, %rdx)		
260(%rcx,%rdx)		
0xFC(,%rcx,4)		
(%rax, %rdx,4)		

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax	0x100	Register
0x104	0xAB	Absolute Addr.
$0x108		
(%rax)		
4(%rax)		
9(%rax, %rdx)		
260(%rcx,%rdx)		
0xFC(,%rcx,4)		
(%rax, %rdx,4)		

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax	0x100	Register
0x104	0xAB	Absolute Addr.
$0x108	0x108	Immediate
(%rax)		
4(%rax)		
9(%rax, %rdx)		
260(%rcx,%rdx)		
0xFC(,%rcx,4)		
(%rax, %rdx,4)		

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax	0x100	Register
0x104	0xAB	Absolute Addr.
$0x108	0x108	Immediate
(%rax)	0xFF	Addr 0x100
4(%rax)		
9(%rax, %rdx)		
260(%rcx,%rdx)		
0xFC(,%rcx,4)		
(%rax, %rdx,4)		

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax	0x100	Register
0x104	0xAB	Absolute Addr.
$0x108	0x108	Immediate
(%rax)	0xFF	Addr 0x100
4(%rax)	0xAB	Addr 0x104
9(%rax, %rdx)		
260(%rcx,%rdx)		
0xFC(,%rcx,4)		
(%rax, %rdx,4)		

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax	0x100	Register
0x104	0xAB	Absolute Addr.
$0x108	0x108	Immediate
(%rax)	0xFF	Addr 0x100
4(%rax)	0xAB	Addr 0x104
9(%rax, %rdx)	0x11	Addr 0x10C
260(%rcx,%rdx)		
0xFC(,%rcx,4)		
(%rax, %rdx,4)		

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax	0x100	Register
0x104	0xAB	Absolute Addr.
$0x108	0x108	Immediate
(%rax)	0xFF	Addr 0x100
4(%rax)	0xAB	Addr 0x104
9(%rax, %rdx)	0x11	Addr 0x10C
260(%rcx,%rdx)	0x13	Addr 0x108
0xFC(,%rcx,4)		
(%rax, %rdx,4)		

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax	0x100	Register
0x104	0xAB	Absolute Addr.
$0x108	0x108	Immediate
(%rax)	0xFF	Addr 0x100
4(%rax)	0xAB	Addr 0x104
9(%rax, %rdx)	0x11	Addr 0x10C
260(%rcx,%rdx)	0x13	Addr 0x108
0xFC(,%rcx,4)	0xFF	Addr 0x100
(%rax, %rdx,4)		

Exercises - 1

- Practice Problem 3-1
- Assume the following values are stored at the indicated memory addresses and registers:

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x104	0xAB	%rcx	0x1
0x108	0x13	%rdx	0x3
0x10C	0x11		

Operand	Value	Comment
%rax	0x100	Register
0x104	0xAB	Absolute Addr.
$0x108	0x108	Immediate
(%rax)	0xFF	Addr 0x100
4(%rax)	0xAB	Addr 0x104
9(%rax, %rdx)	0x11	Addr 0x10C
260(%rcx,%rdx)	0x13	Addr 0x108
0xFC(,%rcx,4)	0xFF	Addr 0x100
(%rax, %rdx,4)	0x11	Addr 0x10C

Exercises - 2

- Practice Problem 3-2
- For each of the following lines of assembly language, determine the appropriate instruction suffix based on the operands. (For example, mov can be written as movb, movw, movl, or movq.)

```
mov__   %eax, (%rsp)
mov__   (%rax), %dx
mov__   $0xFF, %bl
mov__   (%rsp, %rdx,4), %dl
mov__   (%rdx), %rax
mov__   %dx, (%rax)
```

Exercises - 2

- Practice Problem 3-2
- For each of the following lines of assembly language, determine the appropriate instruction suffix based on the operands. (For example, mov can be written as movb, movw, movl, or movq.)

```
mov_l_   %eax, (%rsp)
mov__   (%rax), %dx
mov__   $0xFF, %bl
mov__   (%rsp, %rdx,4), %dl
mov__   (%rdx), %rax
mov__   %dx, (%rax)
```

Exercises - 2

- Practice Problem 3-2
- For each of the following lines of assembly language, determine the appropriate instruction suffix based on the operands. (For example, mov can be written as movb, movw, movl, or movq.)

```
mov_l_   %eax, (%rsp)
mov_w_   (%rax), %dx
mov__    $0xFF, %bl
mov__    (%rsp, %rdx,4), %dl
mov__    (%rdx), %rax
mov__    %dx, (%rax)
```

Exercises - 2

- Practice Problem 3-2
- For each of the following lines of assembly language, determine the appropriate instruction suffix based on the operands. (For example, mov can be written as movb, movw, movl, or movq.)

```
mov_l_   %eax, (%rsp)
mov_w_   (%rax), %dx
mov_b_   $0xFF, %bl
mov__   (%rsp, %rdx,4), %dl
mov__   (%rdx), %rax
mov__   %dx, (%rax)
```

Exercises - 2

- Practice Problem 3-2
- For each of the following lines of assembly language, determine the appropriate instruction suffix based on the operands. (For example, mov can be written as movb, movw, movl, or movq.)

```
mov_l_   %eax, (%rsp)
mov_w_    (%rax), %dx
mov_b_    $0xFF, %bl
mov_b_    (%rsp, %rdx,4), %dl
mov__   (%rdx), %rax
mov__   %dx, (%rax)
```

Exercises - 2

- Practice Problem 3-2
- For each of the following lines of assembly language, determine the appropriate instruction suffix based on the operands. (For example, mov can be written as movb, movw, movl, or movq.)

```
mov_l_   %eax, (%rsp)
mov_w_    (%rax), %dx
mov_b_    $0xFF, %bl
mov_b_    (%rsp, %rdx,4), %dl
mov_q_    (%rdx), %rax
mov__    %dx, (%rax)
```

Exercises - 2

- Practice Problem 3-2
- For each of the following lines of assembly language, determine the appropriate instruction suffix based on the operands. (For example, mov can be written as movb, movw, movl, or movq.)

```
mov_l_   %eax, (%rsp)
mov_w_    (%rax), %dx
mov_b_    $0xFF, %bl
mov_b_    (%rsp, %rdx,4), %dl
mov_q_    (%rdx), %rax
mov_w_    %dx, (%rax)
```

Exercises - 3

- Practice Problem 3-3
- Each of the folllowing lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

```
movb   $0xF, (%ebx)
movl    %rax, (%rsp)
movw    (%rax), 4(%rsp)
movb   %al, %sl
movq %rax, $0x123
movl    %eax, %rdx
movb %si, 8(%rbp)
```

Exercises - 3

- Practice Problem 3-3
- Each of the folllowing lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

```
movb    $0xF, (%ebx)        : Can not use %ebx as address register
movl    %rax, (%rsp)        :
movw    (%rax), 4(%rsp)     :
movb    %al, %sl            :
movq %rax, $0x123           :
movl    %eax, %dx           :
movb %si, 8(%rbp)           :
```

Exercises - 3

- Practice Problem 3-3
- Each of the folllowing lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

```
movb   $0xF, (%ebx)       : Can not use %ebx as address register
movl   %rax, (%rsp)       : Mismatch between instruction suffix and register id
movw    (%rax), 4(%rsp)   :
movb   %al, %sl           :
movq %rax, $0x123         :
movl   %eax, %dx          :
movb %si, 8(%rbp)         :
```

Exercises - 3

- Practice Problem 3-3
- Each of the folllowing lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

```
movb   $0xF, (%ebx)       : Can not use %ebx as address register
movl   %rax, (%rsp)       : Mismatch between instruction suffix and register id
movw    (%rax), 4(%rsp)   : Both of the operands can not be memory
movb   %al, %sl           :
movq %rax, $0x123         :
movl   %eax, %dx          :
movb %si, 8(%rbp)         :
```

Exercises - 3

- Practice Problem 3-3
- Each of the folllowing lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

```
movb   $0xF, (%ebx)        : Can not use %ebx as address register
movl   %rax, (%rsp)        : Mismatch between instruction suffix and register id
movw    (%rax), 4(%rsp)    : Both of the operands can not be memory
movb   %al, %sl            : sl is not a register
movq %rax, $0x123          :
movl   %eax, %dx           :
movb %si, 8(%rbp)          :
```

Exercises - 3

- Practice Problem 3-3
- Each of the folllowing lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

```
movb   $0xF, (%ebx)        : Can not use %ebx as address register
movl   %rax, (%rsp)        : Mismatch between instruction suffix and register id
movw    (%rax), 4(%rsp)    : Both of the operands can not be memory
movb   %al, %sl            : sl is not a register
movq %rax, $0x123          : Immediate value can not be destination operand
movl   %eax, %dx           :
movb %si, 8(%rbp)          :
```

Exercises - 3

- Practice Problem 3-3
- Each of the folllowing lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

```
movb   $0xF, (%ebx)        : Can not use %ebx as address register
movl   %rax, (%rsp)        : Mismatch between instruction suffix and register id
movw    (%rax), 4(%rsp)    : Both of the operands can not be memory
movb   %al, %sl            : sl is not a register
movq %rax, $0x123          : Immediate value can not be destination operand
movl   %eax, %dx           : Size of the destination operand is incorrect
movb %si, 8(%rbp)          :
```

Exercises - 3

- Practice Problem 3-3
- Each of the folllowing lines of code generates an error message when we invoke the assembler. Explain what is wrong with each line.

```
movb   $0xF, (%ebx)        : Can not use %ebx as address register
movl   %rax, (%rsp)        : Mismatch between instruction suffix and register id
movw    (%rax), 4(%rsp)    : Both of the operands can not be memory
movb   %al, %sl            : sl is not a register
movq %rax, $0x123          : Immediate value can not be destination operand
movl   %eax, %dx           : Size of the destination operand is incorrect
movb %si, 8(%rbp)          : Mismatch between instruction suffix and register id
```

Exercises - 4

- Practice Problem 3-5
- You are given the following information. A function with prototype

*void decode1 (long *xp, long *yp, long *zp)*

is compiled into assembly code, yielding the code given or right side.

Parameters xp, yp, and zp are stored in registers %rdi, %rsi, and %rdx, respectively.

Write C code for decode1 that will have an effect equivalent to the assemby code given.

```
void decode1 (long *xp, long *yp, long *zp)
xp in %rdi, yp in %rsi, zp in %rdx
decode1:
movq (%rdi), %r8
movq (%rsi), %rcx
movq (%rdx), %rax
movq %r8, (%rsi)
movq %rcx, (%rdx)
movq %rax, (%rdi)
ret
```

Exercises - 4

void decode1 (long *xp, long *yp, long *zp)

xp in %rdi, yp in %rsi, zp in %rdx

```
decode1:
movq (%rdi), %r8      ; r8=x
movq (%rsi), %rcx     ; rcx=y
movq (%rdx), %rax     ; rax=z
movq %r8, (%rsi)      ; *yp=x
movq %rcx, (%rdx)     ; *zp=y
movq %rax, (%rdi)     ; *xp=z
ret
```

void decode1 (long *xp, long *yp, long *zp)

```
{
        long x = *xp;
        long y = *yp;
        long z = *zp;

        *yp = x;
        *zp = y;
        *xp = z;

}
```

Exercises - 5

- Practice Problem 3-6
- Suppose register %rbx holds value p and %rdx hold value q. Fill the table below with formulas indicating the value that will be stored in register %rax for each of the given assembly code instructions

Instruction	Result
leaq 9(%rdx), %rax	
leaq (%rdx,%rbx), %rax	
leaq (%rdx,%rbx,3), %rax	
leaq (%rbx,%rbx,7), %rax	
leaq 0xE(,%rdx,3), %rax	
leaq 6(%rbx,%rdx,7), %rax	

Exercises - 5

- Practice Problem 3-6
- Suppose register %rbx holds value p and %rdx hold value q. Fill the table below with formulas indicating the value that will be stored in register %rax for each of the given assembly code instructions

Instruction	Result
leaq 9(%rdx), %rax	9+q
leaq (%rdx,%rbx), %rax	
leaq (%rdx,%rbx,3), %rax	
leaq (%rbx,%rbx,7), %rax	
leaq 0xE(,%rdx,3), %rax	
leaq 6(%rbx,%rdx,7), %rax	

%rbx -> p
%rdx -> q

Exercises - 5

- Practice Problem 3-6
- Suppose register %rbx holds value p and %rdx hold value q. Fill the table below with formulas indicating the value that will be stored in register %rax for each of the given assembly code instructions

Instruction	Result
leaq 9(%rdx), %rax	9+q
leaq (%rdx,%rbx), %rax	q+p
leaq (%rdx,%rbx,3), %rax	
leaq (%rbx,%rbx,7), %rax	
leaq 0xE(,%rdx,3), %rax	
leaq 6(%rbx,%rdx,7), %rax	

%rbx -> p
%rdx -> q

Exercises - 5

- Practice Problem 3-6
- Suppose register %rbx holds value p and %rdx hold value q. Fill the table below with formulas indicating the value that will be stored in register %rax for each of the given assembly code instructions

Instruction	Result
leaq 9(%rdx), %rax	9+q
leaq (%rdx,%rbx), %rax	q+p
leaq (%rdx,%rbx,3), %rax	q+3p
leaq (%rbx,%rbx,7), %rax	
leaq 0xE(,%rdx,3), %rax	
leaq 6(%rbx,%rdx,7), %rax	

%rbx -> p
%rdx -> q

Exercises - 5

- Practice Problem 3-6
- Suppose register %rbx holds value p and %rdx hold value q. Fill the table below with formulas indicating the value that will be stored in register %rax for each of the given assembly code instructions

Instruction	Result
leaq 9(%rdx), %rax	9+q
leaq (%rdx,%rbx), %rax	q+p
leaq (%rdx,%rbx,3), %rax	q+3p
leaq (%rbx,%rbx,7), %rax	8p
leaq 0xE(,%rdx,3), %rax	
leaq 6(%rbx,%rdx,7), %rax	

%rbx -> p
%rdx -> q

Exercises - 5

- Practice Problem 3-6
- Suppose register %rbx holds value p and %rdx hold value q. Fill the table below with formulas indicating the value that will be stored in register %rax for each of the given assembly code instructions

Instruction	Result
leaq 9(%rdx), %rax	9+q
leaq (%rdx,%rbx), %rax	q+p
leaq (%rdx,%rbx,3), %rax	q+3p
leaq (%rbx,%rbx,7), %rax	8p
leaq 0xE(,%rdx,3), %rax	3q+14
leaq 6(%rbx,%rdx,7), %rax	

%rbx -> p
%rdx -> q

Exercises - 5

- Practice Problem 3-6
- Suppose register %rbx holds value p and %rdx hold value q. Fill the table below with formulas indicating the value that will be stored in register %rax for each of the given assembly code instructions

Instruction	Result
leaq 9(%rdx), %rax	9+q
leaq (%rdx,%rbx), %rax	q+p
leaq (%rdx,%rbx,3), %rax	q+3p
leaq (%rbx,%rbx,7), %rax	8p
leaq 0xE(,%rdx,3), %rax	3q+14
leaq 6(%rbx,%rdx,7), %rax	7q+p+6

%rbx -> p
%rdx -> q

Exercises - 6

- Practice Problem 3-7
- Consider the following code, in which we have omitted the expressions being computed.

```
short scale3(short x, short y, short z) {
        short t = _____;
        return t;
    }
```

Compiling the actual function with GCC yields the following assembly code:

```
short scale3(short x, short y, short z)
x in %rdi, y in %rsi, z in %rdx
scale3:
    leaq (%rsi, %rsi, 9), %rbx
    leaq (%rbx, %rdx), %rbx
    leaq (%rbx, %rdi, %rsi), %rbx
    ret
```

Fill in the missing expression in the C code

Exercises - 6

short scale3(short x, short y, short z)
x in %rdi, y in %rsi, z in %rdx
scale3:

```
    leaq (%rsi, %rsi, 9), %rbx     ; t = 9y + y =10y
    leaq (%rbx, %rdx), %rbx        ; t = 10y + z
    leaq (%rbx, %rdi, %rsi), %rbx  ; t = (10y + z) + x*y
    ret
```

x -> %rdi
y -> %rsi
z -> %rdx
t -> %rbx

Exercises - 7

- Practice Problem 3-8
- Assume the following values are stored at the indicated memory addresses and registers as in table below.
- According to given information, fill the table on the right side

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x108	0xAB	%rcx	0x1
0x110	0x13	%rdx	0x3
0x118	0x11		

Instruction	Destination	Value
addq %rcx, (%rax)		
subq %rdx, 8(%rax)		
imul $16, (%rax, %rdx, 8)		
incq 16(%rax)		
decq %rcx		
subq %rdx,%rax		

Exercises - 7

- Practice Problem 3-8
- Assume the following values are stored at the indicated memory addresses and registers as in table below.
- According to given information, fill the table on the right side

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x108	0xAB	%rcx	0x1
0x110	0x13	%rdx	0x3
0x118	0x11		

Destination is the address stored in rax
0xFF + rcx = 0xFF+0x1 = 0x100

Instruction	Destination	Value
addq %rcx, (%rax)	0x100	0x100
subq %rdx, 8(%rax)		
imul $16, (%rax, %rdx, 8)		
incq 16(%rax)		
decq %rcx		
subq %rdx,%rax		

Exercises - 7

- Practice Problem 3-8
- Assume the following values are stored at the indicated memory addresses and registers as in table below.
- According to given information, fill the table on the right side

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x108	0xAB	%rcx	0x1
0x110	0x13	%rdx	0x3
0x118	0x11		

Dest. is the address stored in rax + 8
0xAB - rdx = 0xAB- 0x3 = 0xA8

Instruction	Destination	Value
addq %rcx, (%rax)	0x100	0x100
subq %rdx, 8(%rax)	0x108	0xA8
imul $16, (%rax, %rdx, 8)		
incq 16(%rax)		
decq %rcx		
subq %rdx,%rax		

Exercises - 7

- Practice Problem 3-8
- Assume the following values are stored at the indicated memory addresses and registers as in table below.
- According to given information, fill the table on the right side

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x108	0xAB	%rcx	0x1
0x110	0x13	%rdx	0x3
0x118	0x11		

Dest. is the addr. stored in rax +(rdx*8)
0x11 * 16 = = 0x110

Instruction	Destination	Value
addq %rcx, (%rax)	0x100	0x100
subq %rdx, 8(%rax)	0x108	0xA8
imul $16, (%rax, %rdx, 8)	0x118	0x110
incq 16(%rax)		
decq %rcx		
subq %rdx,%rax		

Exercises - 7

- Practice Problem 3-8
- Assume the following values are stored at the indicated memory addresses and registers as in table below.
- According to given information, fill the table on the right side

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x108	0xAB	%rcx	0x1
0x110	0x13	%rdx	0x3
0x118	0x11		

Dest. is the address stored in rax+16
0x13 + 1 = 0x14

Instruction	Destination	Value
addq %rcx, (%rax)	0x100	0x100
subq %rdx, 8(%rax)	0x108	0xA8
imul $16, (%rax, %rdx, 8)	0x118	0x110
incq 16(%rax)	0x110	0x14
decq %rcx		
subq %rdx,%rax		

Exercises - 7

- Practice Problem 3-8
- Assume the following values are stored at the indicated memory addresses and registers as in table below.
- According to given information, fill the table on the right side

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x108	0xAB	%rcx	0x1
0x110	0x13	%rdx	0x3
0x118	0x11		

Destination is rcx
0x1 - 1= 0x0

Instruction	Destination	Value
addq %rcx, (%rax)	0x100	0x100
subq %rdx, 8(%rax)	0x108	0xA8
imul $16, (%rax, %rdx, 8)	0x118	0x110
incq 16(%rax)	0x110	0x14
decq %rcx	%rcx	0x0
subq %rdx,%rax		

Exercises - 7

- Practice Problem 3-8
- Assume the following values are stored at the indicated memory addresses and registers as in table below.
- According to given information, fill the table on the right side

Address	Value	Register	Value
0x100	0xFF	%rax	0x100
0x108	0xAB	%rcx	0x1
0x110	0x13	%rdx	0x3
0x118	0x11		

Destination is rax
rax - rdx = 0x100 - 0x3 = 0xFD

Instruction	Destination	Value
addq %rcx, (%rax)	0x100	0x100
subq %rdx, 8(%rax)	0x108	0xA8
imul $16, (%rax, %rdx, 8)	0x118	0x110
incq 16(%rax)	0x110	0x14
decq %rcx	%rcx	0x0
subq %rdx,%rax	%rax	0xFD

Exercises - 8

- Practice Problem 3-10
- Consider the following code, in which we have omitted the expressions being computed.

```
short arith3(short x, short y, short z)
{
    short p1 = _____;
    short p2 = _____;
    short p3 = _____;
    short p4 = _____;
    return p4;
    }
```

Compiling the actual function with GCC yields the following assembly code:

short arith3(short x, short y, short z)

x in %rdi, y in %rsi, z in %rdx

```
arith3:
    orq %rsi, %rdx
    sarq $9, %rdx
    notq %rdx
    movq %rdx, %rbx
    subq %rsi, %rbx
    ret
```

Fill in the missing expressions in the C code

Exercises - 8

short arith3(short x, short y, short z) {

short p1 = __y|z_ ;

short p2 = _____;

short p3 = _____;

short p4 = _____;

return p4;

}

short arith3(short x, short y, short z)

x in %rdi, y in %rsi, z in %rdx

arith3:

```
orq %rsi, %rdx
sarq $9, %rdx
notq %rdx
movq %rdx, %rbx
subq %rsi, %rbx
ret
```

x -> %rdi

y -> %rsi

z -> %rdx

p4 -> %rbx

Exercises - 8

```
short arith3(short x, short y, short z) {
        short p1 = __y|z__ ;
        short p2 = _p1>>9_;
        short p3 = _____;
        short p4 = _____;
        return p4;
    }
```

```
short arith3(short x, short y, short z)
x in %rdi, y in %rsi, z in %rdx
arith3:
    orq %rsi, %rdx
    sarq $9, %rdx
    notq %rdx
    movq %rdx, %rbx
    subq %rsi, %rbx
    ret
```

x -> %rdi
y -> %rsi
z -> %rdx
p4 -> %rbx

Exercises - 8

```
short arith3(short x, short y, short z) {
        short p1 = __y|z__ ;
        short p2 = _p1>>9_;
        short p3 = __~p2__;
        short p4 = _____;
        return p4;
    }
```

```
short arith3(short x, short y, short z)
x in %rdi, y in %rsi, z in %rdx
arith3:
    orq %rsi, %rdx
    sarq $9, %rdx
    notq %rdx
    movq %rdx, %rbx
    subq %rsi, %rbx
    ret
```

x -> %rdi
y -> %rsi
z -> %rdx
p4 -> %rbx

Exercises - 8

```
short arith3(short x, short y, short z) {
        short p1 = __y|z__ ;
        short p2 = _p1>>9_;
        short p3 = __~p2__;
        short p4 = _p3-y__;
        return p4;
    }
```

```
short arith3(short x, short y, short z)
x in %rdi, y in %rsi, z in %rdx
arith3:
    orq %rsi, %rdx
    sarq $9, %rdx
    notq %rdx
    movq %rdx, %rbx
    subq %rsi, %rbx
    ret
```

x -> %rdi
y -> %rsi
z -> %rdx
p4 -> %rbx

p3 moves to rbx then subtruct rsi

Questions ?